

04 Prove Assignment: Writing Functions

Purpose

Prove that you can write functions with parameters and call those functions with arguments.

Problem Statement

Modern computers are capable of performing all sorts of calculations to produce numbers. However, they are also capable of performing calculations to produce art, illustrations, animations, movies, and music.

Assignment

During the previous lesson's milestone, you wrote code to draw at least the sky, clouds, and ground of an outdoor scene. During this lesson, you will write code that draws the remaining objects in your scene. Your program can draw any outdoor scene that you like as long as it meets these requirements:

1. The scene must be outdoor and include part of the sky.
2. The sky must have clouds.
3. The scene must include repetitive objects, such as blades of grass, trees, leaves on a tree, birds, flowers, insects, fish, pickets in a fence, dashed lines on a road, buildings, bales of hay, snowmen, snowflakes, or icicles.

Your program must be divided into functions such as `draw_sky`, `draw_cloud`, `draw_ground`, `draw_bird`, `draw_flower`, `draw_insect`, `draw_fish`, or

`draw_snowman`. Each repetitive object in your scene should be drawn by a function that your program calls repeatedly, once for each repeated object. For example, your program could include a function named `draw_leaf` that your program repeatedly calls to draw each leaf on a tree at a different location.

As you write your program, write it so that it draws objects in the order of farthest away to nearest. For example, you program should draw the sky first, then clouds, then the ground, then trees, then insects in the trees. Be creative.

Helpful Documentation

- The [prepare content](#) for lesson 2 explains how to call functions.
- The prepare content for the previous lesson explains how to [write functions](#).
- The prepare content for this lesson lesson explains the [properties of a good function](#).
- The [prove milestone](#) of the previous lesson describes this assignment in more detail and contains additional [helpful documentation](#).
- The documentation for the Draw 2D library includes [example programs](#), a [function reference](#), and a table of [supported colors](#).

Testing Procedure

Verify that your program works correctly by following each step in this testing procedure:

1. Run your program and verify that it correctly opens a window and draws within that window a complete outdoor scene that contains the sky, clouds, the ground, and other elements.

Exceeding the Requirements

If you wish to exceed the requirements of this assignment, you could write code to add more items to your scene. The most efficient way to add more items to your scene is to write a function that draws something, such as `draw_pebble`, `draw_snowman`, or `draw_fish` and then call that function many times in your `draw_scene` function. Exceeding the requirements of this assignment is optional.

Ponder

After you finish this assignment, congratulate yourself because you wrote a Python program with many user-defined functions. As you wrote your program, what did you learn about organizing a program into functions? Did you learn that you can work more efficiently by writing a function once and calling it many times with different arguments?

Submission

To submit your program, return to I-Learn and do these two things:

1. Upload your program (the .py file) for feedback.
2. Add a submission comment that specifies the grading category that best describes your program along with a one or two sentence justification for your choice. The grading criteria are:
 - a. Some attempt made
 - b. Developing but significantly deficient
 - c. Slightly deficient
 - d. Meets requirements
 - e. Exceeds requirements